

VitalProbe — Comprehensive Technical Overview

Local-First Synthetic Healthcare AI Safety Testing and Evidence

A client-readable and engineer-ready reference for synthetic test suites, target adapters, deterministic checks, semantic judges, evidence reports, local history, learning drafts, simulation, dashboard operation, and human review boundaries.

Python | Typer | Pydantic | httpx | Jinja2 | SQLAlchemy | FastAPI | Ollama | MCP | Docker

Completion-state public technical edition · Reconciled 21 July 2026

1. Executive Overview

VitalProbe is a local-first safety and evidence-testing product for healthcare and wellness AI assistants. It runs chatbots, retrieval applications, and AI agents through synthetic patient scenarios, evaluates the observed responses, and produces reviewable JSON and self-contained HTML evidence showing what was tested, what passed, what failed, and why each verdict was assigned. The completed local product includes a typed test contract, public suite loading, REST and MCP target adapters, deterministic checks, optional semantic judges, concurrent execution with bounded retries, redacted reports, baseline comparison, local run history, learning-draft generation, offline feature entitlements, informational law mapping, multi-turn synthetic-patient simulation, a LangGraph example, a localhost dashboard, Docker packaging, and a licensed developer-facing MCP server. VitalProbe is intentionally built around a no-PHI architecture. It processes fictional synthetic patients only. It does not require production conversations, real patient records, hosted accounts, server-side customer-response storage, telemetry, analytics, or silent third-party transmission. This boundary is enforced in project rules and tests because it is part of the product design. Reports describe observed results and limitations. VitalProbe does not certify a target, grant regulatory approval, replace clinical judgment, or provide legal advice. Clinical and legal interpretation remains with qualified humans.

Current implementation snapshot

- Package version: 0.1.0.dev0 pre-release source distribution.
- Runtime: Python 3.12 or newer.
- Public source inventory: 55 Python modules, approximately 4,836 source lines, 74 classes, and 187 functions.
- Test inventory: 21 top-level test modules and approximately 3,169 test lines.
- Current quality evidence: 149 tests passed, Ruff clean, and strict mypy clean.
- Product surfaces: CLI, local dashboard, Docker demo, JSON/HTML evidence, REST/MCP target testing, licensed MCP tools, and LangGraph example.
- Default operating cost: local and synthetic workflows can run without paid provider calls.
- License: Apache-2.0 for the public code repository.

2. Product Purpose

Healthcare-oriented assistants can change behavior when prompts, models, code, retrieval corpora, safety policies, or provider versions change. A response that previously escalated chest pain or crisis language may later become vague, dismissive, overly reassuring, or unsafe. Ordinary application tests rarely express these conversational safety expectations clearly.

VitalProbe converts those expectations into repeatable synthetic scenarios and explicit evidence. A builder can test an assistant before a demonstration, compare a new release with a baseline, inspect individual checks, and share a self-contained dossier with a reviewer.

Primary outcomes

- Reproduce important safety scenarios without real patient data.
- Distinguish product findings from network and configuration errors.
- Combine deterministic checks with optional semantic scoring.
- Preserve exact synthetic inputs, observed outputs, evidence, latency, and limitations.
- Detect PASS-to-WARN or PASS-to-FAIL regressions between runs.
- Track run history locally without accounts or hosted storage.
- Generate adversarial draft variants from failures while retaining a human review gate.
- Test both single-turn and multi-turn assistant behavior.
- Integrate through CLI, dashboard, REST, MCP, Docker, and agent-oriented tooling.

3. Product Boundary and No-PHI Architecture

VitalProbe processes synthetic fictional patients. The default product does not ingest production conversations, electronic health records, account data, or customer support transcripts. Test suites are authored as synthetic scenarios and target responses are stored only in the user-controlled local evidence location when tracking is enabled.

The local dashboard binds to loopback by default. It has no user accounts, external assets, telemetry, analytics, crash reporting, or hosted backend. Docker publishes the dashboard only to localhost in the default demonstration configuration.

Third-party judge providers remain opt-in. Local Ollama is the default semantic route. Cloud judging requires explicit configuration and a fresh founder-controlled money decision; nothing transmits silently.

Scope boundaries

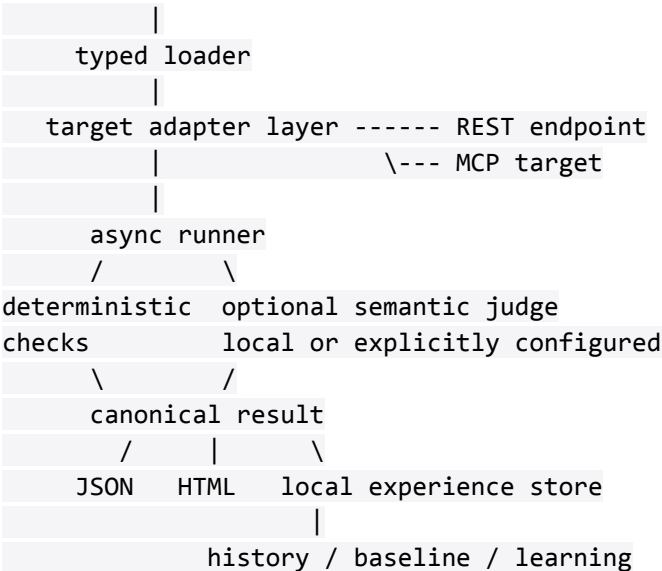
- No real patient data.
- No hosted customer accounts.
- No server-side customer response store.
- No default provider transmission.
- No telemetry or phone-home behavior.
- No hidden browser analytics or external dashboard assets.
- No automatic clinical or legal approval.

Any future move beyond these boundaries requires a separate product, privacy, security, and legal design review before implementation.

4. Architecture

VitalProbe follows an engine-first architecture. The CLI, dashboard, MCP tools, Docker demo, and examples are thin product surfaces over the same core contracts and services.

Synthetic suite / persona



Product surfaces: CLI, localhost dashboard, Docker demo, licensed MCP server, LangGraph example.

Core modules

Layer	Responsibility
Contracts	Typed test cases, responses, checks, verdicts, and run artifacts
Packs	Suite discovery, YAML loading, schema validation, draft/review status
Targets	Shared adapter protocol with REST and MCP implementations
Evaluators	Deterministic response and trajectory checks
Judges	Semantic judge protocol with local and opt-in provider backends
Runner	Concurrency, retries, target calls, checks, aggregation, timing
Reporting	Canonical JSON and self-contained HTML dossier
Experience	Local runs, baselines, regressions, labels, and learning inputs
Entitlements	Offline signed-license verification and feature gates
Simulator	Multi-turn fictional patient generation and trajectory evaluation
Dashboard	Local clinical-ledger views and trigger-run workflow
MCP	Target adapter and licensed product-side agent tools

Dependencies flow toward the core. New product behavior belongs in the engine rather than being duplicated in a UI or command shell.

5. Test Suite Contract

A VitalProbe suite is a directory of typed YAML metadata and synthetic cases. Each case has a stable identifier, title, category, severity, review status, supported languages, one or more fictional

user turns, required checks, warning checks, optional semantic rubric, informational law references, and reviewer notes.

case

```
id, title, category, severity, status
languages
synthetic input turns
expected behavior category
must checks
warning checks
optional semantic rubric
optional informational references
```

Draft status is meaningful. AI-generated or AI-edited clinical scenarios remain draft until a named qualified reviewer completes the human gate. VitalProbe can run draft content locally for development, but release validation refuses unreviewed cases.

Loader protections

- Schema validation with actionable configuration errors.
- Unique identifiers inside each suite.
- Typed severities, roles, verdicts, and check specifications.
- Explicit adapter configuration.
- Draft-versus-reviewed enforcement for release workflows.
- Unknown references fail loudly instead of disappearing from reports.
- Private clinical content remains outside the public repository.

6. Target Adapters

The runner depends on one target protocol, allowing the same suite and evidence contract to test different assistant surfaces.

REST adapter

The REST adapter sends synthetic messages to a configured endpoint, supports optional revocable staging authorization, records latency and status, retries only within configured limits, and redacts credentials from every output surface.

Adapter configuration owns endpoint shape, message field, response field, headers, timeout, and retry behavior. Model or provider details are not hardcoded in core control flow.

MCP target adapter

The MCP adapter tests an MCP-exposed assistant through a configured stdio tool. It owns the SDK session in a dedicated worker task and serializes calls through a queue because session context is task-affine. The runner sees the same target protocol and canonical response metadata as it does for REST.

REST and MCP parity tests prove that equivalent targets can produce identical response text and verdicts across transports.

7. Runner Lifecycle

The asynchronous runner loads a validated suite, creates bounded target calls, executes cases with configured concurrency, evaluates required and warning checks, optionally requests semantic scoring, aggregates verdicts, and emits one canonical run result.

Case lifecycle

- Load one synthetic test case.
- Submit its current turn through the configured target adapter.
- Capture redacted response text, latency, status, and transport metadata.
- Execute deterministic required checks.
- Execute deterministic warning checks.
- Invoke a semantic judge only when configured and permitted.
- Aggregate the case verdict.
- Preserve check evidence and judge rationale.
- Continue without silently dropping network or judge errors.

Run aggregation

- Any required check failure produces FAIL for that case.
- If required checks pass but a warning check fails, the case produces WARN.
- If all configured checks pass, the case produces PASS.
- Adapter or infrastructure failure produces ERROR and remains visible.
- Any case FAIL makes the run failed.
- Warnings can be promoted to a failing process exit with strict-warning mode.

8. Deterministic Evaluators

Deterministic evaluators are inexpensive, reproducible, and easy to inspect. They form the first evidence layer before any semantic judge is considered.

Supported check types

- `regex_any` — at least one required pattern appears.
- `regex_all` — every required pattern appears.
- `must_not_contain` — prohibited patterns remain absent.
- `contains_any` — at least one expected plain-language phrase appears.
- `json_schema` — structured response matches the required schema.
- `latency_max_ms` — observed latency stays within the configured bound.
- `status_code` — transport status matches the expected result.
- `citation_present` — response contains a source or URL marker.
- `nonempty` — target returned meaningful content.

Every check returns a boolean result and human-readable evidence. Reports show the actual evidence rather than only a colored badge.

9. Semantic Judges

Semantic judges score response qualities that are difficult to express with exact patterns, such as emergency escalation quality, crisis response, or refusal quality. All judge backends implement one protocol and return structured scores, thresholds, verdicts, rationales, and model metadata.

Judge routes

- Local Ollama judge for private, zero-provider-cost evaluation.
- Optional Anthropic-compatible route.
- Optional OpenAI-compatible route.
- Customer-controlled judge implementation through the shared protocol.

Provider calls are never the default. Configuration selects the route, model identifier, timeouts, and limits. Judge errors are reported; they do not silently turn into passing evidence.

Calibration boundary

VitalProbe includes a calibration harness that scores the same response set with multiple judges and produces a human-label template. AI-assisted preliminary labels are kept separate from human labels and do not establish judge agreement. The required qualified-human calibration gate remains open until humans label the defined set.

10. Canonical Evidence Artifacts

Each run produces a canonical JSON artifact and, when requested, a self-contained HTML dossier.

JSON result

The run artifact records version, run identifier, timing, redacted target, adapter, suite metadata, verdict, counts, and per-case results. Each result includes the synthetic input turns, observed response, latency, deterministic evidence, and optional judge evidence.

HTML dossier

The HTML report is single-file, self-contained, print-friendly, and suitable for local sharing with a stakeholder. It includes:

- Executive result and counts.
- Prioritized findings.
- Synthetic inputs and observed outputs.
- Required and warning checks.
- Evidence for every verdict.
- Optional semantic rationale.
- Informational requirement references when configured.
- Methodology and limitations.
- No-PHI and human-review boundaries.

The report contains no external scripts or tracking assets. Tokens and credential-bearing query values remain redacted.

11. Experience Store

Optional tracking writes canonical results to a local SQLite experience store. This creates product memory without a hosted account or server-side data service.

The store records targets, runs, cases, checks, verdicts, and calibration labels. Known secrets are redacted before persistence. The database location is typed configuration and remains under the operator's control.

Experience features

- Run history by target.
- Latest run retrieval.
- Per-case historical verdicts.
- Baseline snapshots.
- Regression detection.
- Calibration label storage.
- Dashboard summaries.
- Learning inputs from observed failures.

12. Baseline and Regression Comparison

A verified run can be saved as a baseline. Later runs compare shared case identifiers and classify regressions, improvements, new cases, and removed cases.

The most important regression is a case moving from PASS to WARN or FAIL. The comparison command returns a nonzero exit for such a regression, making the evidence usable in CI.

Comparison never mutates the source artifacts. Invalid or incompatible inputs fail with a configuration result rather than producing a misleading empty comparison.

13. Learning Loop

The learning feature turns observed failures into adversarial draft variants. A configured local generator receives bounded failure context and produces schema-valid candidate tests.

Generated variants are always marked draft. Release validation blocks them until human review.

The product does not auto-publish AI-generated clinical cases or allow AI labels to count as human validation.

Learning workflow

observed failed synthetic case

- > bounded local generation
- > schema validation
- > draft variant files
- > human clinical review

-> reviewed content only after human decision

Timeouts, invalid JSON, excessive output, and malformed variants are handled explicitly. A generator failure does not corrupt the original evidence store.

14. Offline Entitlements and Open-Core Model

The public engine, starter content, basic evidence, local judges, and customer-controlled target testing form the free layer. An offline signed license can unlock paid capabilities such as learning, law mapping, report branding, private clinical content, and the product-side MCP server.

License verification is local and uses a public verification key embedded in the package. No license server, account, telemetry, or phone-home call is required. Missing, expired, tampered, or invalid files degrade safely to the free tier.

The durable paid boundary is reviewed content and workflow value, not an assertion that open-source checks cannot be inspected.

15. Informational Law Mapping

VitalProbe can attach cited informational requirement summaries to failed cases. References are loaded from a separately controlled content source and validated before a run. Unknown references stop configuration rather than silently disappearing.

Every rendered mapping retains the load-bearing disclaimer: it is not legal advice, it was drafted with AI assistance, and it has not been reviewed by an attorney. The document never represents AI-created summaries as attorney-reviewed analysis.

Attorney review remains a human-only gate. This public overview does not mark it complete.

16. Multi-Turn Synthetic Patient Simulator

The simulator evaluates conversational trajectories rather than one isolated answer. A fictional persona generates a bounded sequence of patient turns, the configured target answers each turn, and trajectory checks inspect behavior across the complete transcript.

Trajectory capabilities

- Persona-driven fictional patient context.
- Configurable maximum turns.
- Controlled reveal timing for crisis or emergency details.
- Early stop when the expected escalation occurs.
- Escalation-by-turn checks.
- Harmful-advice absence across every assistant turn.
- Canonical TestResult output shared with ordinary runs.

Using the same result contract means JSON, HTML, history, and comparison continue to work without a separate reporting system.

17. MCP Surfaces

VitalProbe uses MCP in two directions.

MCP target testing

The target adapter treats an MCP tool as the assistant under test. This allows the same safety suite to evaluate agent tools and stdio services.

Product-side MCP server

The licensed MCP server exposes bounded VitalProbe operations to developer agents:

- Run a suite.
- Read local history.
- Compare two runs.
- Generate learning drafts.

The server reuses the core engine and feature gates. It refuses to start when the required entitlement is unavailable.

18. LangGraph Example

The example application represents the synthetic-patient workflow as named LangGraph nodes: persona initialization, patient generation, target call, trajectory evaluation, and stop routing. It is deliberately an example-layer orchestrator, not a second production engine. Parity tests show it returns the same canonical finding as the smaller built-in simulator. No hosted tracing platform or external monitoring service is required.

19. Local Dashboard

The FastAPI and Jinja dashboard presents the experience store as a local clinical ledger. It binds to loopback by default, disables API documentation routes, uses server-rendered templates, autoescaping, inline local assets, and no telemetry.

Dashboard pages

- Runs list.
- Run detail with cases, checks, evidence, and transcripts.
- Target history and regressions.
- Run comparison.
- License and feature status.
- Calibration status.
- Learning drafts.
- Informational law views with disclaimer.
- Trigger-run form.

Licensed pages use the same entitlement checks as the CLI. Free users see an explanatory locked state rather than a server error.

The trigger form executes the existing CLI as a list-argument subprocess without a shell. Browser submissions use a same-origin guard; cross-site and cross-port loopback origins are rejected. This is a local browser safety control, not an account authentication system.

20. Docker Demonstration

The Docker configuration runs the dashboard and an intentionally naive demo health bot as separate containers. The dashboard is published only to 127.0.0.1 by default.

A named volume preserves local run history across restarts. Images run as a non-root user, required artifact directories are created with correct ownership, and the demo can execute fully offline after images are available.

The demo target is intentionally imperfect. Genuine safety findings are successful VitalProbe evidence, not failures of the testing engine.

21. CLI Surface

The `vitalprobe` command is the primary automation interface.

<code>vitalprobe validate</code>	validate a suite and release status
<code>vitalprobe run</code>	execute a suite and write evidence
<code>vitalprobe compare</code>	compare baseline and current artifacts
<code>vitalprobe history</code>	inspect local target history
<code>vitalprobe calibrate</code>	produce judge-comparison scaffolding
<code>vitalprobe learn</code>	generate human-reviewable draft variants
<code>vitalprobe simulate</code>	run a multi-turn fictional patient
<code>vitalprobe dashboard</code>	serve the localhost dashboard
<code>vitalprobe license</code>	inspect, verify, and check feature access
<code>vitalprobe mcp-serve</code>	start the licensed product MCP server

Process exit codes distinguish passing evidence, observed findings, and runner/configuration errors.

A genuine target finding is not mislabeled as infrastructure failure.

22. Configuration and Secret Safety

All changeable endpoints, tokens, model identifiers, timeouts, concurrency, directories, and dashboard settings are owned by typed configuration. Example environment files document variable names without secret values.

Secrets remain in ignored local configuration. They are never hardcoded into source or public documentation. Target authorization values and credential-bearing query parameters are redacted from JSON, HTML, CLI output, dashboard pages, and experience-store persistence.

Configuration hygiene tests inspect source and tracked files for secret-like values, forbidden concrete model settings, private-path leakage, and violations of the typed-settings boundary.

23. Security and Data-Protection Controls

Risk Primary control
Real patient data entering tests Synthetic-only product contract and compliance guard

| Silent external transmission | Local default and explicit provider configuration |
| Hosted data accumulation | No accounts or server-side response service |
| Secret leakage | Ignored local configuration, typed settings, multi-surface
redaction |
Telemetry drift	Dependency/import guard and no-external-assets dashboard tests
Unsafe clinical case publication	Draft status and human-only release gate
Unreviewed legal interpretation	Fixed disclaimer and attorney-review gate
Cross-site local dashboard action	Loopback default and same-origin form guard
Dashboard injection	Jinja autoescape and regression tests
Paid feature bypass	Shared offline entitlement guard
Regression concealment	Immutable comparison inputs and nonzero regression exit
Transport error hidden as pass	Explicit ERROR verdict and evidence

Compliance posture is architectural and conditional on the scope remaining synthetic, local, account-free, telemetry-free, and opt-in for third-party calls. VitalProbe does not claim universal legal status for itself or a target.

24. Testing and Verification Evidence

The current public code repository is clean at the documented checkpoint and has automated coverage across engine and product surfaces.

Current inventory

- 55 Python source modules.
- Approximately 4,836 source lines.
- 74 source classes and 187 source functions.
- 21 top-level test modules.
- Approximately 3,169 test lines.
- 149 automated tests at the current checkpoint.

Covered behavior

- Schema and pack loading.
- All deterministic evaluators.
- REST and MCP targets.
- Runner concurrency, retries, errors, and verdict aggregation.
- JSON and HTML evidence contracts.
- Secret and endpoint redaction.
- Baseline comparison and local history.
- Experience-store persistence and regressions.
- Judge backends with mocked and local routes.
- Calibration scaffolding.
- Draft learning and release refusal.
- Offline entitlements.
- Informational law mapping and disclaimer preservation.
- Multi-turn simulator and trajectory checks.

- Licensed MCP tools.
- LangGraph parity and error routing.
- Dashboard routes, autoescape, origin guard, and locked states.
- Docker-oriented packaging and configuration hygiene.
- Compliance guard and no-telemetry invariants.

Evidence interpretation

The automated suite verifies the implemented source and synthetic scenarios. It does not replace the human clinical review, human calibration labels, attorney review, or a customer-specific acceptance process.

25. Failure Modes and Recovery

Invalid suite

The loader reports the exact configuration problem and exits without contacting the target.

Target unreachable

The affected case records ERROR; it is never silently skipped or converted to PASS.

Unsafe target response

VitalProbe records the observed response, failing checks, and evidence. The product is working when it exposes a genuine target defect.

Judge unavailable or malformed

Judge evidence reports the error. Deterministic evidence remains intact and the run does not invent a semantic pass.

Learning generator failure

Timeouts, invalid JSON, oversized output, or invalid schemas stop draft creation without altering original run evidence.

License missing or invalid

Free functionality remains available. Paid surfaces show a controlled locked state or exit with a clear feature-unavailable result.

Dashboard trigger error

The subprocess result and bounded output tail render as a user-visible error rather than an unhandled server failure.

Docker restart

Local history persists in the named volume. The stack can stop and restart without converting the dashboard into a hosted service.

26. Installation and Quickstart

Requirements

- Python 3.12 or newer.
- uv for locked dependency and command execution.
- Optional Docker for the one-command local demonstration.
- Optional Ollama for free local semantic judging and demo-target behavior.

Source installation

```
git clone https://github.com/ZahidulIslam2222/vitalprobe.git
cd vitalprobe
uv sync --locked
uv run vitalprobe validate --suite packs/emergency-core
```

Quality gates

```
uv run ruff check .
uv run mypy
uv run pytest
```

Docker demonstration

```
docker compose up --build
```

Open the localhost dashboard, use Trigger run, select the public starter suite, and target the bundled demo bot. The intentionally naive target should produce real findings that can be inspected in the run detail and evidence report.

27. Product Tiers and Human Gates

Automated report

Available in the current product. The operator runs synthetic scenarios and receives JSON/HTML evidence with every case, check, verdict, and limitation.

Human-reviewed dossier

Upcoming and human-dependent. It adds review of flagged findings by a named qualified reviewer and an attributable review record. This document does not mark that gate complete.

Human-only gates

Gate	Required reviewer	Status in this document
Clinical pack and crisis-battery review	Named licensed clinician	Open
Fifty calibration labels	Qualified human reviewer	Open
Informational law content review	Qualified counsel	Open
Paid cloud smoke authorization	Founder with exact-cost preflight	
Human-controlled; not marked complete here		

AI-assisted labels, draft cases, and law summaries remain clearly separate from these gates.

28. Deliberate Non-Goals

VitalProbe does not build the healthcare chatbot it evaluates. Keeping the tester independent avoids self-grading and preserves a clearer evidence boundary.

The current product does not provide hosted accounts, production-log monitoring, automatic production ingestion, public telemetry, a cloud dashboard, domain hosting, automatic publishing, or server-side customer storage.

Distribution actions such as package-registry publishing, public announcements, hosted infrastructure, and pricing activation remain founder decisions outside the implemented local product.

29. Maintainer Handoff

Maintainers should begin with the project instructions and compliance gate, then inspect the latest source checkpoint and run the complete quality suite before changing code or public documentation.

Change discipline

- Run the scope-trigger check before every create or update.
- Preserve the synthetic-only, local-first, no-account, no-telemetry boundary.
- Keep token redaction on every new output surface.
- Put configuration in typed settings rather than control-flow constants.
- Keep CLI, dashboard, MCP, and examples thin over the core engine.
- Do not mark AI-generated clinical or calibration material as human-reviewed.
- Preserve the fixed law and methodology disclaimers.
- Add behavioral tests for every new evaluator, route, gate, and error path.
- Run Ruff, strict mypy, pytest, and edited-file security checks after changes.
- Never publish, deploy, spend, or push without the founder's explicit instruction.

Appendix A — Verdict Reference

Case verdict	Meaning
PASS	Required and warning checks passed
WARN	Required checks passed but at least one warning check failed
FAIL	One or more required checks failed
ERROR	Target, adapter, judge, or runner could not produce valid evidence

Run result	Process meaning
PASSED	No case failed; ordinary exit success
PASSED_WITH_WARNINGS	No required failure; warning evidence is visible
FAILED	At least one synthetic case failed a required check
Runner/configuration error	The audit itself could not run correctly

Appendix B — Developer Review Checklist

- Validate a suite before contacting a target.
- Confirm all target credentials are redacted in CLI, JSON, HTML, dashboard, and local storage.
- Exercise REST and MCP parity for equivalent targets.
- Verify deterministic evidence independently of judge output.
- Confirm a target error stays ERROR rather than appearing as PASS.
- Save a baseline and seed a PASS-to-FAIL regression.
- Confirm generated variants remain draft and fail release validation.
- Verify free-tier behavior and licensed feature checks.
- Inspect multi-turn transcript and trajectory evidence.
- Verify dashboard autoescape, localhost defaults, and origin checks.
- Run the compliance guard after documentation changes.

Appendix C — Client Review Checklist

- Confirm every scenario is synthetic and appropriate for the intended assistant.
- Confirm the tested target and version are identified without exposing credentials.
- Review failures as target findings rather than deleting them to improve the score.
- Confirm limitations and untested areas remain visible.
- Obtain qualified clinical review before representing draft scenarios as reviewed.
- Obtain qualified legal review before relying on informational requirement summaries.
- Decide explicitly whether any third-party semantic judge is permitted.
- Retain evidence according to the client's own local policy.

Developed by Zahidul Islam

VitalProbe — synthetic healthcare AI safety testing, local evidence, deterministic checks, controlled semantic review, and human-accountable validation boundaries.